2.3 Oracle 팰릿 분석

개요

Oracle 팰릿은 Composable Finance 생태계의 핵심 가격 피드 인프라로, 외부 데이터 소스로부터 자산 가격 정보를 수집하고 집계하여 온체인에서 신뢰할 수 있는 가격 데이터를 제공합니다. 이 팰릿은 오프체인 워커(Offchain Worker)를 활용하여 외부 API로부터 가격 데이터를 가져오고, 다중 오라클 시스템을 통해 데이터의 신뢰성과 정확성을 보장합니다.

주요 구성 요소

2.3.1 팰릿 구조

Oracle 팰릿은 다음과 같은 주요 모듈로 구성됩니다:

- **lib.rs**: 메인 팰릿 로직 및 extrinsic 함수들
- **validation.rs**: 입력 데이터 검증 로직
- **weights.rs**: 가중치 계산 및 성능 최적화
- **benchmarking.rs**: 벤치마킹 및 성능 측정

2.3.2 핵심 데이터 구조

AssetInfo

```
#[derive(Encode, Decode, MaxEncodedLen, Default, Debug, PartialEq, Eq, Clone, Ty
pub struct AssetInfo<Percent, BlockNumber, Balance> {
pub threshold: Percent,
```

```
pub min_answers: u32,
pub max_answers: u32,
pub block_interval: BlockNumber,
pub reward_weight: Balance,
pub slash: Balance,
pub emit_price_changes: bool,
}
```

PrePrice

```
#[derive(Encode, Decode, MaxEncodedLen, Clone, Copy, Default, Debug, PartialEq,
pub struct PrePrice<PriceValue, BlockNumber, AccountId> {
pub price: PriceValue,
pub block: BlockNumber,
pub who: AccountId,
}
```

RewardTracker

```
pub struct RewardTracker<Balance, Moment> {
pub period: Moment,
pub start: Moment,
pub total_already_rewarded: Balance,
pub current_block_reward: Balance,
```

```
pub  total_reward_weight: Balance,

}
```

2.3.3 스토리지 구조

Prices

```
#[pallet::storage]

#[pallet::getter(fn prices)]

pub  type Prices<T: Config> = StorageMap<

_,

Blake2_128Concat,

T::AssetId,

Price<T::PriceValue, T::BlockNumber>,

ValueQuery,

>;
```

- **목적**: 각 자산의 최종 집계된 가격 정보 저장
- **키**: `T::AssetId`
- **값**: `Price<PriceValue, BlockNumber>` 구조체

PrePrices

```
#[pallet::storage]
```

```
#[pallet::getter(fn pre_prices)]
pub type PrePrices<T: Config> = StorageMap<
_,
Blake2_128Concat,
T::AssetId,
BoundedVec<PrePrice<T::PriceValue, T::BlockNumber, T::AccountId>, T::MaxPrePrice
ValueQuery,
>;
```

- **목적**: 집계 전 개별 오라클이 제출한 가격 데이터 임시 저장
- **키**: `T::AssetId`
- **값**: `PrePrice` 구조체의 제한된 벡터

PriceHistory

```
#[pallet::storage]
#[pallet::getter(fn price_history)]
pub type PriceHistory<T: Config> = StorageMap<
_,
Blake2_128Concat,
T::AssetId,
BoundedVec<Price<T::PriceValue, T::BlockNumber>, T::MaxHistory>,
ValueQuery,
>;
```

- **목적**: TWAP(Time-Weighted Average Price) 계산을 위한 가격 이력 저장
- **키**: `T::AssetId`
- **값**: 가격 이력의 제한된 벡터

AssetsInfo

```
#[pallet::storage]
#[pallet::getter(fn asset_info)]
pub  type AssetsInfo<T: Config> = StorageMap<
_,
Blake2_128Concat,
T::AssetId,
AssetInfo<Percent, T::BlockNumber, BalanceOf<T>>,
OptionQuery,
>;
```

- **목적**: 각 자산의 오라클 설정 정보 저장
- **키**: `T::AssetId`
- **값**: `AssetInfo` 구조체

OracleStake

```
#[pallet::storage]
#[pallet::getter(fn oracle_stake)]
```

```
pub type OracleStake<T: Config> = StorageMap<_, Blake2_128Concat, T::AccountId,
```

- **목적**: 각 오라클의 스테이킹된 자산 추적
- **키**: 오라클 계정 ID
- **값**: 스테이킹된 잔액

SignerToController / ControllerToSigner

```
#[pallet::storage]
#[pallet::getter(fn signer_to_controller)]
pub type SignerToController<T: Config> = StorageMap<_, Blake2_128Concat, T::Acc
```

```
#[pallet::storage]
#[pallet::getter(fn controller_to_signer)]
pub type ControllerToSigner<T: Config> = StorageMap<_, Blake2_128Concat, T::Acc
```

- **목적**: 오라클 서명 키와 컨트롤러 키 간의 매핑
- **기능**: 키 분리를 통한 보안 강화

2.3.4 주요 Extrinsic 함수들

add_asset_and_info

```
#[pallet::call_index(0)]
#[pallet::weight(T::WeightInfo::add_asset_and_info())]
```

```
pub fn add_asset_and_info(
origin: OriginFor<T>,
asset_id: T::AssetId,
threshold: Validated<Percent, ValidThreshold>,
min_answers: Validated<u32, ValidMinAnswers<T::MinAnswerBound>>,
max_answers: Validated<u32, ValidMaxAnswer<T::MaxAnswerBound>>,
block_interval: Validated<T::BlockNumber, ValidBlockInterval<T::StalePrice>>,
reward_weight: BalanceOf<T>,
slash: BalanceOf<T>,
emit_price_changes: bool,
) -> DispatchResultWithPostInfo
```

- **목적**: 새로운 자산의 오라클 설정 추가
- **매개변수**:
- `threshold` : 정확도 임계값 (리워드 지급 기준)
- `min_answers` : 집계를 위한 최소 응답 수
- `max_answers` : 집계할 최대 응답 수
- `block_interval` : 가격 업데이트 간격
- `reward_weight` : 리워드 가중치
- `slash` : 부정확한 답변에 대한 슬래시 금액
- **검증**: 모든 매개변수는 `Validated` 타입으로 사전 검증됨
- **이벤트**: `AssetInfoChange`

set_signer

```
#[pallet::call_index(1)]

#[pallet::weight(T::WeightInfo::set_signer())]

pub fn set_signer(

origin: OriginFor<T>,

who: T::AccountId,

signer: T::AccountId,

) -> DispatchResultWithPostInfo
```

- **목적**: 오라클 서명자 설정 및 최소 스테이킹
- **프로세스**:

1. 컨트롤러와 서명자 중복 확인
2. 최소 스테이킹 금액 예치
3. 매핑 관계 설정

- **이벤트**: `SignerSet`, `StakeAdded`

submit_price

```
#[pallet::call_index(6)]

#[pallet::weight((T::WeightInfo::submit_price(T::MaxAnswerBound::get()), Dispatc

pub fn submit_price(

origin: OriginFor<T>,

price: T::PriceValue,

asset_id: T::AssetId,
```

```
) -> DispatchResultWithPostInfo
```

- **목적**: 오라클이 가격 데이터 제출
- **특징**: Operational 클래스로 분류되어 우선 처리
- **검증사항**:
- 충분한 스테이킹 잔액 확인
- 중복 제출 방지
- 최대 응답 수 제한
- **프로세스**:

1. 스테이킹 잔액 및 슬래시 가능 금액 확인
2. `PrePrices` 에 가격 데이터 추가
3. `AnswerInTransit` 에 슬래시 예약 금액 추가

- **이벤트**: `PriceSubmitted`
- **수수료**: 성공 시 가스 수수료 면제 (`Pays::No`)

adjust_rewards

```
#[pallet::call_index(2)]
#[pallet::weight(T::WeightInfo::adjust_rewards())]
pub fn adjust_rewards(
origin: OriginFor<T>,
annual_cost_per_oracle: BalanceOf<T>,
num_ideal_oracles: u8,
```

```
) -> DispatchResultWithPostInfo
```

- **목적**: 오라클 리워드 시스템 조정
- **계산 로직**:

```
let new_annual_reward = annual_cost_per_oracle.saturating_mul(num_ideal_oracles
let remaining_blocks_in_year = period_left.checked_div(T::MsPerBlock::get())?;
let current_block_reward = (new_annual_reward - total_already_rewarded)
.checked_div(&remaining_blocks_in_year)?;
```

- **이벤트**: `RewardingAdjustment`

2.3.5 스테이킹 시스템

add_stake

```
#[pallet::call_index(3)]
#[pallet::weight(T::WeightInfo::add_stake())]
pub fn add_stake(origin: OriginFor<T>, stake: BalanceOf<T>) -> DispatchResultW
```

- **목적**: 기존 오라클의 스테이킹 증가

remove_stake

```
#[pallet::call_index(4)]
#[pallet::weight(T::WeightInfo::remove_stake())]
```

```
pub fn remove_stake(origin: OriginFor<T>) -> DispatchResultWithPostInfo
```

- **목적**: 스테이킹 해제 요청 (지연 해제)
- **프로세스**:

1. 현재 스테이킹 금액 확인
2. 해제 블록 번호 계산 (`current_block + StakeLock`)
3. `DeclaredWithdraws` 에 해제 요청 저장
4. 즉시 스테이킹 상태에서 제거

reclaim_stake

```
#[pallet::call_index(5)]
#[pallet::weight(T::WeightInfo::reclaim_stake())]
pub fn reclaim_stake(origin: OriginFor<T>) -> DispatchResultWithPostInfo
```

- **목적**: 지연 기간 후 스테이킹 자산 회수
- **조건**: `current_block > unlock_block`
- **프로세스**:

1. 해제 가능 시점 확인
2. 예약된 자산 해제
3. 컨트롤러 계정으로 자산 전송
4. 매핑 관계 제거

2.3.6 오프체인 워커 (Offchain Worker)

check_requests 함수

```
fn offchain_worker(_block_number: T::BlockNumber) {
log::info!("Offchain worker triggered");
Self::check_requests();
}
```

외부 API 호출

오프체인 워커는 HTTP 요청을 통해 외부 가격 데이터를 수집합니다:

```
use sp_runtime::offchain::{http, Duration};

// HTTP 요청 예시
let request = http::Request::get("https://api.example.com/price")
.deadline(Duration::from_millis(3000))
.send()
.map_err(|_| "HTTP request failed")?;
```

서명된 트랜잭션 제출

```
impl<T: Config> Pallet<T> {
fn submit_signed_price(price: T::PriceValue, asset_id: T::AssetId) -> Result<()
let signer = Signer::<T, T::AuthorityId>::all_accounts();
```

```
if !signer.can_sign() {
return Err("No local accounts available");
}
let results = signer.send_signed_transaction(|_account| {
Call::submit_price { price, asset_id }
});
for (acc, res) in &results {
match res {
Ok(()) => log::info!("Price submitted by {:?}", acc.id),
Err(e) => log::error!("Failed to submit price: {:?}", e),
}
}
Ok(())
}
}
```

2.3.7 가격 집계 시스템

가격 집계 로직

```
fn update_prices_and_rewards(block: T::BlockNumber) -> Weight {
let mut weight = Weight::zero();
for (asset_id, asset_info) in AssetsInfo::<T>::iter() {
```

```
if block % asset_info.block_interval == T::BlockNumber::zero() {
let pre_prices = PrePrices::<T>::get(asset_id);
if pre_prices.len() >= asset_info.min_answers as usize {
if let Ok(aggregated_price) = Self::aggregate_prices(&pre_prices) {
Self::update_price(asset_id, aggregated_price, block);
Self::handle_payout(&pre_prices, aggregated_price, asset_id, &asset_info);
}
PrePrices::<T>::remove(asset_id);
weight = weight.saturating_add(T::WeightInfo::aggregate_prices());
}
}
}
weight
}
```

가격 집계 알고리즘

현재 구현에서는 중앙값(median) 기반 집계를 사용합니다:

```
fn aggregate_prices(
pre_prices: &[PrePrice<T::PriceValue, T::BlockNumber, T::AccountId>]
) -> Result<T::PriceValue, DispatchError> {
let mut prices: Vec<T::PriceValue> = pre_prices.iter().map(|p| p.price).collec
prices.sort();
```

```
let len = prices.len();
let median = if len % 2 == 0 {
(prices[len / 2 - 1] + prices[len / 2]) / 2.into()
} else {
prices[len / 2]
};
Ok(median)
}
```

2.3.8 리워드 및 슬래시 시스템

정확도 기반 리워드

```
fn handle_payout(
pre_prices: &[PrePrice<T::PriceValue, T::BlockNumber, T::AccountId>],
price: T::PriceValue,
asset_id: T::AssetId,
asset_info: &AssetInfo<Percent, T::BlockNumber, BalanceOf<T>>,
) -> DispatchResult {
for answer in pre_prices {
let accuracy: Percent = if answer.price < price {
PerThing::from_rational(answer.price, price)
} else {
```

```
let adjusted_number = price.saturating_sub(answer.price - price);
PerThing::from_rational(adjusted_number, price)
};
if accuracy < asset_info.threshold {
// 슬래시 처리
Self::slash_oracle(&answer.who, asset_info.slash, asset_id);
} else {
// 리워드 처리
Self::reward_oracle(&answer.who, asset_id);
}
}
Ok(())
}
```

슬래시 메커니즘

```
fn slash_oracle(who: &T::AccountId, slash_amount: BalanceOf<T>, asset_id: T::As
let new_stake = Self::oracle_stake(who)
.unwrap_or_default()
.saturating_sub(slash_amount);
OracleStake::<T>::insert(who, new_stake);
let _ = T::Currency::repatriate_reserved(
who,
```

```
&T::TreasuryAccount::get(),
slash_amount,
BalanceStatus::Free,
);
Self::deposit_event(Event::UserSlashed(who.clone(), asset_id, slash_amount));
}
```

리워드 분배

```
fn reward_oracle(who: &T::AccountId, asset_id: T::AssetId) {
if let Some(reward_tracker) = Self::get_reward_tracker_if_enabled() {
let asset_reward_weight = Self::asset_info(asset_id)
.map(|info| info.reward_weight)
.unwrap_or_default();
let reward_amount = reward_tracker.current_block_reward
.saturating_mul(asset_reward_weight)
.checked_div(&reward_tracker.total_reward_weight)
.unwrap_or_default();
if !reward_amount.is_zero() {
let _ = T::Currency::deposit_creating(who, reward_amount);
Self::deposit_event(Event::OracleRewarded(who.clone(), asset_id, reward_amount))
}
}
```

```
}
```

2.3.9 TWAP (Time-Weighted Average Price) 시스템

TWAP 계산

```
impl<T: Config> Oracle for Pallet<T> {
fn get_twap_for_amount(
asset_id: Self::AssetId,
amount: Self::Balance,
) -> Result<Self::Balance, DispatchError> {
let prices_length = Self::price_history(asset_id).len();
let twap_window: usize = <Self as Oracle>::TwapWindow::get().into();
if twap_window > prices_length + 1 {
Self::get_price(asset_id, amount).map(|p| p.price)
} else {
let price = Self::get_twap(asset_id, twap_window)?;
Self::quote(asset_id, price, amount)
}
}
}
```

가격 이력 관리

```
fn update_price_history(asset_id: T::AssetId, price: Price<T::PriceValue, T::Bl
PriceHistory::<T>::mutate(asset_id, |history| {
if history.len() >= T::MaxHistory::get() as usize {
history.remove(0); // 가장 오래된 가격 제거
}
let _ = history.try_push(price);
});
}
```

2.3.10 검증 시스템

입력 검증

validation.rs 모듈에서 다양한 검증 로직을 제공합니다:

ValidThreshold

```
impl Validate<Percent, ValidThreshold> for ValidThreshold {
fn validate(input: Percent) -> Result<Percent, &'static str> {
if input >= Percent::from_percent(100) {
return Err("INVALID_THRESHOLD")
}
Ok(input)
}
```

```
}
```

ValidMinAnswers

```
impl<MinAnswer, MinAnswerBound> Validate<MinAnswer, ValidMinAnswers<MinAnswerBou
for ValidMinAnswers<MinAnswerBound>
where
MinAnswerBound: Get<MinAnswer>,
MinAnswer: Zero + PartialEq + Eq + Ord + PartialOrd,
{
fn  validate(input: MinAnswer) -> Result<MinAnswer, &'static str> {
if  input < MinAnswerBound::get() {
return Err("INVALID_MIN_ANSWERS")
}
Ok(input)
}
}
```

ValidBlockInterval

```
impl<BlockInterval, StalePrice> Validate<BlockInterval, ValidBlockInterval<Stale
for ValidBlockInterval<StalePrice>
where
StalePrice: Get<BlockInterval>,
```

```
BlockInterval: PartialOrd,
{
fn  validate(input: BlockInterval) -> Result<BlockInterval, &'static str> {
if  input <= StalePrice::get() {
return Err("INVALID_BLOCK_INTERVAL_LENGTH")
}
Ok(input)
}
}
```

2.3.11 이벤트 시스템

주요 이벤트들

PriceSubmitted

```
PriceSubmitted(T::AccountId, T::AssetId, T::PriceValue)
```

- 오라클이 가격을 제출했을 때 발생

PriceChanged

```
PriceChanged(T::AssetId, T::PriceValue)
```

- 자산의 집계된 가격이 변경되었을 때 발생

OracleRewarded

```
OracleRewarded(T::AccountId, T::AssetId, BalanceOf<T>)
```

- 정확한 가격을 제출한 오라클에게 리워드가 지급될 때 발생

UserSlashed

```
UserSlashed(T::AccountId, T::AssetId, BalanceOf<T>)
```

- 부정확한 가격을 제출한 오라클이 슬래시될 때 발생

2.3.12 보안 및 검증

주요 보안 메커니즘

1. **스테이킹 요구사항**:

```
ensure!(author_stake >= T::MinStake::get(), Error::<T>::NotEnoughStake);
```

2. **중복 제출 방지**:

```
if  current_prices.iter().any(|candidate| candidate.who == who) {
return Err(Error::<T>::AlreadySubmitted.into())
}
```

3. **최대 응답 수 제한**:

```
if  current_prices.len() as u32 >= asset_info.max_answers {
```

```
return Err(Error::<T>::MaxPrices.into())
}
```

4. **슬래시 예약 시스템**:

```
AnswerInTransit::<T>::mutate(&who, |transit| {
*transit = Some(transit.unwrap_or_default().saturating_add(asset_info.slash));
});
```

경제적 보안

- **스테이킹 기반 참여**: 오라클은 최소 스테이킹 금액을 예치해야 함
- **정확도 기반 인센티브**: 정확한 가격 제출 시 리워드, 부정확한 제출 시 슬래시
- **지연된 스테이킹 해제**: 악의적 행동 후 즉시 탈출 방지

2.3.13 성능 최적화

가중치 기반 실행

- 각 함수는 실제 실행 비용에 따른 가중치 할당
- Operational 클래스 트랜잭션으로 우선 처리

배치 처리

- 블록별로 모든 자산의 가격 업데이트를 일괄 처리
- 효율적인 스토리지 접근 패턴

메모리 효율성

- `BoundedVec` 사용으로 메모리 사용량 제한
- 오래된 데이터 자동 정리

이 Oracle 팰릿은 Composable Finance 생태계에서 신뢰할 수 있는 가격 데이터를 제공하는 핵심 인프라로, 탈중앙화된 오라클 네트워크를 통해 외부 데이터의 온체인 통합을 안전하고 효율적으로 수행합니다.